

Транзакційна модель монетизації (1% комісії)

Технічна специфікація реалізації

1. Концепція монетизації

Монетизація маркетплейсу Sellora базується на **транзакційній моделі**, а не на підписках. Замість обмеження доступу до функціоналу (Free/Pro версії, ліміти на створення товарів або rate-limiting ендпоінтів), платформа застосовує підхід "Win-Win" — монетизація відбувається лише тоді, коли продавець успішно реалізує товар.

Платформа відраховує **1% від фінальної ціни БУДЬ-ЯКОЇ успішної покупки** (як стандартного замовлення типу `REGULAR`, так і групового `GROUP_BUY`).

Ця модель реалізована через сервісні хуки (Service-Level Hooks / Interceptors) на етапі підтвердження оплати та завершення життєвого циклу замовлення або сесії групової покупки.

2. Механізм роботи сервісних хуків (Service-Level Hooks)

Замість того, щоб ускладнювати бізнес-логіку контролерів, логіка утримання комісії винесена у транзакційні методи сервісного шару (зокрема, в `PaymentService`).

Механізм працює за наступним сценарієм:

- Ініціалізація платежу:** Користувач сплачує повну суму товару через платіжний шлюз (LiqPay) або підтверджує післяплату (якщо передбачено системою).
- Обробка Webhook:** Шлюз надсилає асинхронний запит на `/api/v1/payments/webhook`.
- Хук статусу PAID:** `PaymentService` фіксує оплату.

4. **Виконання спліт-розрахунку:** Коли замовлення успішно оплачене, транзакційний хук автоматично:

- Вираховує 1% комісії платформи з фінальної суми замовлення.
- Записує дві транзакції в базу даних (PostgreSQL): виплату мерчанту (99%) та дохід платформи (1%).
- У випадку групових покупок, хук обробляє весь пул замовлень сесії після її успішного закриття.

Цей процес відбувається в рамках однієї ACID-транзакції бази даних, що гарантує фінансову консистентність (неможливість завершення оплати без розрахунку комісії).

3. Технічна реалізація (Код)

Наразі логіка фіксації замовлень та завершення сесії вже існує в нашому `PaymentService.java`. Для впровадження монетизації ми додаємо метод `processMonetizationSplit`, який викликається всередині існуючої транзакції.

3.1. Драфт коду (PaymentService.java)

```
@Service
@RequiredArgsConstructor
public class PaymentService {

    // Існуючі залежності...
    private final MerchantLedgerRepository merchantLedgerRepository;
    private final PlatformRevenueRepository platformRevenueRepository;

    @Transactional
    public void processLiqPayWebhook(String idempotencyKey, Long orderId, String status) {
        // ... існуюча логіка збереження TransactionEvent ...

        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new RuntimeException("Замовлення не знайдено"));

        order.setPaymentStatus(orderPaymentStatus);
        orderRepository.save(order);

        // --- ЛОГІКА ДЛЯ ГРУПОВИХ ПОКУПОК ---
        if ("GROUP_BUY".equals(order.getPurchaseType()) && "PAID".equals(orderPaymentStatus)) {
            if (!groupMemberRepository.existsBySessionIdAndUserId(order.getSessionId(),
                // Додавання учасника...
                GroupBuySession session = groupBuySessionRepository.findById(order.getSessionId());
                int currentMembers = groupMemberRepository.countBySessionId(session.getId());

            if (currentMembers >= session.getLockedTargetSize()) {
                session.setStatus("COMPLETED");
                groupBuySessionRepository.save(session);

                List<Order> sessionOrders = orderRepository.findBySessionId(session.getId());
                for (Order o : sessionOrders) {
                    if ("WAITING_FOR_GROUP".equals(o.getPaymentStatus())) {
                        o.setPaymentStatus("PENDING");
                    }
                }
                orderRepository.saveAll(sessionOrders);

                // ХУК для всієї групи
                processMonetizationSplit(sessionOrders, session.getId());
            }
        }
    }
}
```

```

    }
    // ---- ЛОГІКА ДЛЯ ЗВИЧАЙНИХ ПОКУПОК АБО ІНДИВІДУАЛЬНИХ ПЛАТЕЖІВ ----
    else if ("PAID".equals(orderPaymentStatus) && !"GROUP_BUY".equals(order.getPurc
        // ХУК для індивідуального замовлення
        processMonetizationSplit(List.of(order), null);
    }
}

/**
 * Сервісний хук для спліт-монетизації.
 * Розраховує 1% комісії для кожного замовлення.
 */
private void processMonetizationSplit(List<Order> orders, Long sessionId) {
    for (Order o : orders) {
        if (!"PAID".equals(o.getPaymentStatus()) && !"PENDING".equals(o.getPaymentS
            continue;
        }

        BigDecimal finalPrice = o.getFinalPrice();

        // Розрахунок часток (1% Sellora, 99% Продавець)
        BigDecimal platformCommission = finalPrice.multiply(BigDecimal.valueOf(0.01
            .setScale(2, RoundingMode.HALF_UP);
        BigDecimal merchantPayout = finalPrice.subtract(platformCommission);

        // Збереження доходу платформи
        PlatformRevenue platformRevenue = new PlatformRevenue();
        platformRevenue.setSourceType(sessionId != null ? "GROUP_BUY_COMMISSION" :
        platformRevenue.setSourceId(o.getId()); // прив'язка до конкретного замовле
        platformRevenue.setAmount(platformCommission);
        platformRevenueRepository.save(platformRevenue);

        // Фіксація балансу мерчанта для подальшого виводу (Payout)
        MerchantLedger ledgerEntry = new MerchantLedger();
        ledgerEntry.setMerchantId(o.getMerchantId());
        ledgerEntry.setOrderId(o.getId());
        ledgerEntry.setAmount(merchantPayout);
        ledgerEntry.setType("CREDIT");
        merchantLedgerRepository.save(ledgerEntry);
    }
}
}

```

4. Переваги такого підходу

1. **Відмовостійкість (Resilience):** Обчислення комісії відбувається асинхронно від дій користувача (під час обробки Webhook'а). Це означає, що клієнтський інтерфейс ніколи не блокується важкими фінансовими транзакціями.
2. **Універсальність:** Завдяки єдиному хуку, 1% коректно утримується з **будь-якого** типу транзакції, незалежно від того, чи це звичайна покупка, чи складна групова сесія.
3. **Атомарність (Atomicity):** Завдяки анотації `@Transactional` на рівні методу `processLiqPayWebhook`, зміна статусу оплати та розрахунок комісії гарантовано відбудуться або разом, або не відбудуться взагалі у разі збою.
4. **Масштабованість:** PostgreSQL чудово справляється з записом таких подій у таблиці-леджеру (`merchant_ledger`), що дозволяє легко агрегувати місячні виплати продавцям через окремі job'и (Quartz або Spring Scheduler).